

Pokemon- Datenbankprojekt

Von der Projektplanung
bis zur SQL-Auswertung

Team: Robert, Joel, Berkay, Felix, David
Joel hat den Großteil der Umsetzung betreut und
fehlt heute.

Schwerpunkt
sauberes Daten-
modell, sinnvolle
Normalisierung und
nachvollziehbare
Abfragen

Agenda

1. Projektplanung

Rollen
MUSS / SOLL /
KANN
Vorgehen im
Zeitraumen

2. Datenbankstruktur

Tabellen
Kardinalitäten
Normalisierung

3. Abfragen

JOIN
GROUP BY
SELF JOIN und
HAVING

4. Nächster Durchlauf

Datenlücken
technische
Verbesserungen
Ausblick

Das Projekt in 30 Sekunden

21

Karten in der
Haupttabelle

29

Attacken in der
n:m-Verknüpfung

8

Pokemon-Typen

6

Raritätsstufen

Zielbild

Eine kleine, aber sauber strukturierte Pokemon-Datenbank, in der Pflichtdaten direkt abrufbar sind und typische SQL-Techniken demonstriert werden können.

Ergebnis

Vollständige Pflichtattribute, mehrere verknüpfte Hilfstabellen und genug Datenmaterial für Statistik-, Detail- und Beziehungsabfragen.

Projektplanung und Rollen

Rollenverteilung

Robert	Projektplanung, Einleitung, Zeitrahmen erklären
Joel	Technische Hauptumsetzung, Datenimport, SQLverine-Struktur
Berkay	Datenmodell, Tabellen, Kardinalitäten
Felix	Abfragen 1 und 2 erklären
David	Abfragen 3 und 4, Reflexion, Ausblick

Ablauf in 4 Phasen

- 1 **Analyse**
Rohdaten und Pflichtfelder klären
- 2 **Modell**
Tabellen und Schlüssel festlegen
- 3 **SQL**
Import, Joins, Testabfragen
- 4 **Finish**
Fehler prüfen und präsentieren

Wichtig: Pflichtfelder zuerst, früh testen, Zusatzdaten danach.

MUSS, SOLL, KANN

MUSS

Name
Kartennummer
Typ
HP
Attacken

Diese Felder mussten für jede Karte direkt auswertbar sein.

SOLL

Illustrator
Rarität
Evolution

Diese Infos machen die DB deutlich interessanter, waren aber nicht ganz so kritisch wie die Kern-daten.

KANN

Preis
Schwäche
Resistenz
Rückzugskosten

Hier haben wir bewusst priorisiert, damit die Grundstruktur zuerst stabil steht.

Datenbasis und Importpfad

Vom Rohdatensatz zur relationalen Struktur

Rohdaten

CSV



Zerlegung

Typ, Seltenheit, Attacken



SQLverine

SQLite-DB

Der Rohdatensatz war textlastig. Deshalb wurden wiederkehrende Angaben ausgelagert und Attacken über eine Zwischentabelle modelliert statt als Freitext stehen zu lassen.

21

Zeilen in card

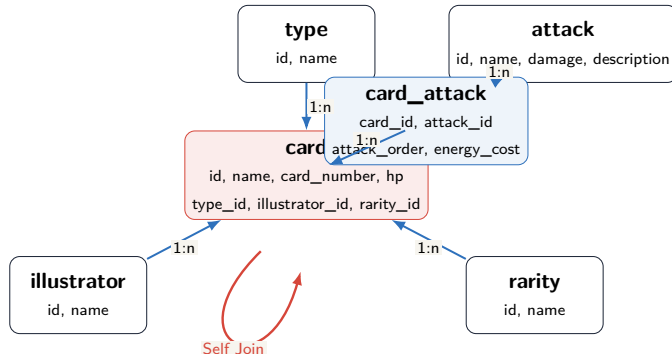
29

Verknüpfungen in
card_attack

5

echte Evolutions-Links

Relationale Struktur der Datenbank



Kerngedanken

- Typ, Illustrator und Rarität werden mehrfach genutzt und deshalb ausgelagert.
- Mehrere Attacken werden über **card_attack** statt über feste Spalten gespeichert.
- Self Join für Evolutionen.

Normalisierung und bekannte Lücken

Normalisierung bringt hier direkt etwas

- type, rarity, illustrator: keine redundanten Textkopien in jeder Zeile
- card_attack: saubere n:m-Beziehung statt mehrere Attackenspalten
- evolves_from_card_id: Entwicklung als Relation statt nur als Freitext

Technisch fehlen noch echte FOREIGN KEY-, NOT NULL- und CHECK-Constraints.

Vollständigkeit der Daten

Name 21/21



Typ 21/21



HP 21/21



Resist. 1/21



Preis 0/21



Evo-FK 5/9



8/14

Abfrage 1: Gesamtansicht einer Karte

SELECT

```
c.card_number,  
c.name,  
t.name AS type,  
c.hp,  
i.name AS illustrator,  
r.name AS rarity,  
GROUP_CONCAT(  
    a.name || ' [' || ca.energy_cost || ']',  
    ' | '  
) AS attacks
```

FROM card c

JOIN type t **ON** t.id = c.type_id

JOIN illustrator i **ON** i.id = c.illustrator_id

JOIN rarity r **ON** r.id = c.rarity_id

LEFT JOIN card_attack ca **ON** ca.card_id = c.id

LEFT JOIN attack a **ON** a.id = ca.attack_id

GROUP BY c.id

ORDER BY c.id;

Pokemon-Datenbankprojekt

Was zeigt die Abfrage?

Alle Pflichtfelder plus wichtige Zusatzinfos in einer einzigen Ansicht.

Beispiel: Camerupt

028/217

Feuer, 140 HP

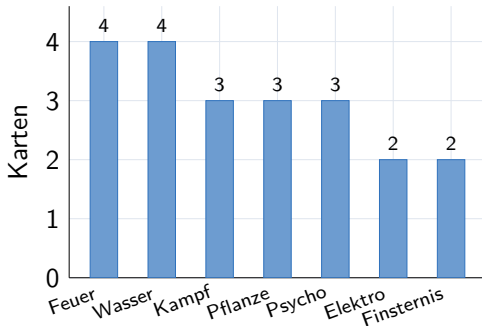
Illustrator: Minahamu

Rarität: Uncommon

Attacken: Bratendes Brennen,
Kraftstamper

Abfrage 2: Welche Typen kommen wie oft vor?

```
SELECT
  t.name AS type,
  COUNT(*) AS cards,
  ROUND(AVG(c.hp), 0) AS avg_hp
FROM card c
JOIN type t ON t.id = c.type_id
GROUP BY t.name
ORDER BY cards DESC, t.name;
```



Feuer und Wasser sind am häufigsten vertreten.
Feuer hat zugleich den höchsten durchschnittlichen HP-Wert.

Abfrage 3: Evolutionen per Self Join

```
SELECT
  c.name AS card,
  e.name AS evolves_from
FROM card c
LEFT JOIN card e
  ON e.id = c.evolves_from_card_id
WHERE c.evolves_from_card_id IS NOT NULL
ORDER BY c.id;
```

Fachlich wichtig

Im Rohdatensatz gibt es 9 Entwicklungskarten. Davon sind nur 5 als echte Fremdschlüssel verknüpft, weil mehrere Vorstufen gar nicht in der Auswahl enthalten sind.

Beispiele

Digda → Digdri

Camaub → Camerupt

Gelatini → Gelatropo → Gelatwino

Owei → Kokowei

Abfrage 4: Karten mit genau zwei Attacks

```
SELECT
  c.name,
  COUNT(*) AS attacks,
  c.hp
FROM card c
JOIN card_attack ca
  ON ca.card_id = c.id
GROUP BY c.id
HAVING COUNT(*) = 2
ORDER BY c.hp DESC, c.name;
```

Trefferliste

Ns Gelatwino	Ns Gelatroppo
Camerupt	Erikas Ultrigaria
Team Rockets	Banette
Kokowei	Ns Flampion
Team Rockets Digdri	

8 von 21 Karten besitzen genau zwei Attacks. Gerade hier sieht man, warum card_attack als Zwischentabelle flexibler ist als feste Spalten wie attack1 und attack2.

Was würden wir beim nächsten Mal verbessern?

Datenmodell

- Set-Tabelle ergänzen statt nur Kartennummern zu speichern
- Preis als eigenes Attribut oder Verlaufstabelle ergänzen
- Alle Evolutionsvorstufen vollständig importieren
- Unbenutzte Tabelle `card_new` bereinigen oder sinnvoll einsetzen

Technik: echte FOREIGN KEY- und NOT NULL-Constraints, Importskript und feste Smoke Tests.

Unser Fazit

Die Datenbank ist für ein Schulprojekt schon gut nutzbar: Pflichtdaten sind vollständig, Joins funktionieren und die Kernbeziehungen sind nachvollziehbar modelliert.

Der nächste professionelle Schritt wäre aber klar:

- mehr Vollständigkeit in den Zusatzdaten
- mehr technische Sicherheit im Schema
- mehr Automatisierung beim Import

Kleine Datenbasis, aber saubere Idee.

Vielen Dank

Fragen?



Grundlage: `information/Haupt.db`, Rohdaten-CSV und eigene SQL-Abfragen